

Hardness, Equivalence, and Quantum Annealing

Why these problems are hard, why one encoding serves them all, and the first quantum machine that attacks it

Lecture 4

September 14, 2026

Where we are

L1-L3 built the encoding.

- L1-L2: model a COP as instance, feasible set, cost function
- L3: solve one classically — greedy, DP, approximation
- L3: every model can be pushed into Ising / QUBO form

Two questions were left open.

- Why is brute force the only exact method that always works?
- If the answer is the ground state of a Hamiltonian, can a machine *find* that ground state by letting physics do the work?

This lecture, in three moves

1. **Hardness** — what NP-hardness does and does not claim
2. **Equivalence** — why one encoding, QUBO, serves all of them
3. **Machines** — adiabatic evolution and quantum annealing, the first hardware built for this encoding

P, NP and NP-hardness*

- **P**: decision problems solvable in polynomial time
- **NP**: decision problems whose *yes* answers can be *verified* in polynomial time
 - “Is there a tour of cost ≤ 80 ?” : hand me the tour and I check it in $O(n)$
 - verifying is easy; *finding* is the hard part
- **NP-complete**: in NP, and every problem in NP reduces to it
- **NP-hard**: at least as hard as every problem in NP, but need not be in NP itself
 - the *optimization* versions of TSP, Knapsack and Max-Cut are NP-hard
 - their *decision* versions (“is there a solution of value $\geq k$?”) are NP-complete

■ The problem of $P = NP$ is open. Nobody has proved that these problems require exponential time; nobody has found a polynomial algorithm for any of them, and that if one existed for a single NP-complete problem it would exist for all of them.

* Slide with is not included in the assessment

Problem reductions*

- A **reduction** from problem A to problem B is a polynomial-time transformation turning any instance of A into an instance of B , such that solving B solves A
- Written $A \leq_p B$: “ B is at least as hard as A ”
- Reductions do two jobs in this course, and they are opposite in spirit:
 - **Pessimistic use**: to prove hardness. If a known NP-hard problem reduces to yours, yours is NP-hard too
 - **Constructive use**: to *reuse a solver*. If your problem reduces to QUBO, then anything that solves QUBO solves your problem
- The second use is the one that matters from here on

Takeaway: The same fact “everything reduces to everything” is bad news for complexity and very good news for engineering. It means one machine can be pointed at all of them.

A reduction, worked: Hamiltonian Cycle $\leq_p$ TSP*

- **Hamiltonian Cycle:** given a graph $G = (V, E)$ on n vertices, is there a cycle visiting every vertex exactly once? (NP-complete)
- **The construction.** Build a *complete* weighted graph on the same vertices, and price the edges by whether G had them:

$$w(i, j) = \begin{cases} 1, & (i, j) \in E \\ 2, & (i, j) \notin E \end{cases}$$

- Every tour visits n edges, so its cost is at least n , with equality exactly when *every* edge it uses came from E

$G \text{ has a Hamiltonian cycle} \iff \min_{\pi} c(\pi) = n$

- The transformation is $O(n^2)$ — so a polynomial TSP solver would give a polynomial Hamiltonian Cycle solver. **Hence TSP is NP-hard**
- Delete edge $(3, 4)$ from G and the cheapest tour costs $5 > n$: every tour is now forced onto at least one weight-2 edge, and the reduction reports “no cycle”

G — edge $(2, 4)$ is missing



complete graph, weights 1 / 2



Tours: $1234 \rightarrow 4$, $1243 \rightarrow 5$, $1324 \rightarrow 5$.

$\min = 4 = n$, so G is Hamiltonian.

Every problem we modelled is NP-hard*

Problem	Status	Note
TSP (optimization)	NP-hard	decision version NP-complete
0/1 Knapsack	NP-hard	but pseudo-polynomial via DP (L3)
Max-Cut	NP-hard	even on unweighted graphs
Graph colouring	NP-hard	3-colouring already NP-complete
Job scheduling $P C_{\max}$	NP-hard	NP-hard already for $m = 2$ machines
Vehicle routing (VRP)	NP-hard	contains TSP as a special case

- Knapsack is the instructive exception: NP-hard, yet L3's DP solved it exactly

Equivalence: one target for all of them*

- Because every problem in NP reduces to every NP-complete problem, the choice of which one to build a machine for is an *engineering* decision, not a mathematical one
- The community settled on **QUBO**, equivalently, the Ising model, and the reasons are physical, not mathematical:
 - it is *quadratic*: two-body terms map onto two-qubit couplings
 - it is *unconstrained*: no feasibility bookkeeping in hardware
 - it is *diagonal* in the computational basis: the cost of a bit string is read straight off a measurement
- Lucas (2014) writes down explicit Ising formulations for all 21 of Karp's NP-complete problems — the standard reference for “how do I encode *my* problem?”

Takeaway: Build one machine that minimizes $x^T Q x$, and in principle you have a machine for every problem in this course.

A. Lucas (2014). “Ising formulations of many NP problems.” *Frontiers in Physics* 2, art. 5. arXiv:1302.5843

QUBO and Ising

QUBO: binary variables $x \in \{0, 1\}^n$

$$\min_{x \in \{0,1\}^n} x^T Q x = \sum_i Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j$$

The diagonal carries the linear terms,
because $x_i^2 = x_i$.

- One change of variable connects them: $s_i = 2x_i - 1$

$$Q_{ij} = -4J_{ij}, \quad Q_{ii} = 2 \sum_{j \in N(i)} J_{ij} - 2h_i, \quad \text{constant} = \sum_i h_i - \sum_{(i,j) \in E} J_{ij}$$

- The constant shifts every energy equally — it changes the reported energy, never the argmin

Ising: spins $s \in \{-1, +1\}^n$

$$H(s) = - \sum_{(i,j) \in E} J_{ij} s_i s_j - \sum_{i \in V} h_i s_i$$

Ground state = the answer.

Constraints become penalties

- QUBO is *unconstrained*. Every constraint from L2 must be folded into the objective:

$$\min_y c(y) \quad \text{s.t.} \quad g(y) = 0 \quad \longrightarrow \quad \min_y c(y) + \lambda g(y)^2$$

Constraint	Penalty term	Appears in
$\sum_i y_i = 1$ (one-hot)	$\lambda(\sum_i y_i - 1)^2$	TSP, assignment, colouring
$y_i + y_j \leq 1$	$\lambda y_i y_j$	independent set, scheduling
$\sum_i w_i y_i \leq W$	$\lambda(\sum_i w_i y_i + s - W)^2$	knapsack (needs slack bits)

- Choosing λ is a real design decision**
 - too small: the QUBO's optimum is infeasible for the original problem
 - too large: the penalty dominates, the landscape flattens, and the solver can no longer see the objective it is meant to minimize

Worked: the L3 knapsack as a QUBO

- Items $(w_i, v_i) = (2, 3), (3, 4), (4, 5)$, capacity $W = 5$. Maximize value, so minimize $-\sum_i v_i x_i$
- The capacity constraint is an *inequality*, so it needs a **slack variable** $s \in \{0, \dots, 5\}$ to turn it into an equality $\sum_i w_i x_i + s = W$
- s itself must be binary-encoded: $s = s_0 + 2s_1 + 4s_2$, so $\lceil \log_2(W+1) \rceil = 3$ extra bits

$$f(x, s) = -(3x_1 + 4x_2 + 5x_3) + \lambda (2x_1 + 3x_2 + 4x_3 + s_0 + 2s_1 + 4s_2 - 5)^2$$

- Expanding the square produces all the quadratic terms — this is a QUBO in 6 binary variables
- **3 items cost 6 qubits.** The slack encoding is where qubit counts explode: capacity W costs $\lceil \log_2 W \rceil$ extra qubits, whatever n is

The catch: Inequality constraints need slack, slack needs qubits, and qubits are the scarcest resource on every machine in this lecture.

The classical baseline you have to beat

- Before claiming anything for a quantum method, know what a laptop already does

Method	What it is	Status on QUBO/Ising
Simulated annealing	thermal hill-climbing with a cooling schedule	the default baseline; very hard to beat
Tabu / large-neighbourhood search	local search with memory	state of the art on many QUBOs
Branch and bound (Gurobi, CPLEX)	exact, with pruning	exact answers on surprisingly large instances
Semidefinite relaxation (Goemans-Williamson)	SDP + randomized rounding	0.8786 guarantee for Max-Cut

Will quantum computing settle $P = NP$?

- **No. This is not a matter of waiting for better hardware**
- The class of problems solvable efficiently by a quantum computer is **BQP**
 - BQP is not believed to contain NP-complete problems
 - No quantum algorithm is known that solves an NP-complete problem in polynomial time, and there are good reasons to expect none exists
- Grover's algorithm searches N items in $O(\sqrt{N})$ — a *quadratic* speedup, and provably optimal for unstructured search
 - $\sqrt{2^n} = 2^{n/2}$ is still exponential
- Shor's algorithm is exponential-speedup, but factoring is *not* NP-complete — it sits in NP without being known to be NP-complete or in P

Takeaway: The realistic hope is not a polynomial algorithm for NP-hard problems. It is better heuristics — faster, or better-quality answers, on instances that matter.

Adiabatic evolution and quantum annealing

Letting physics find the ground state

From cost function to Hamiltonian

- Promote each classical spin $s_i \in \{-1, +1\}$ to the Pauli operator σ_i^z :

$$H_P = - \sum_{(i,j) \in E} J_{ij} \sigma_i^z \sigma_j^z - \sum_{i \in V} h_i \sigma_i^z$$

- H_P is **diagonal** in the computational basis: $H_P |z\rangle = c(z) |z\rangle$ for every bit string z
 - its 2^n eigenvalues are exactly the 2^n costs of the classical problem
 - its ground state is the computational basis state encoding the optimum
 - measuring in the z basis returns a candidate solution, already readable
- So the optimization problem has become a physics question: *prepare the ground state of H_P*

The adiabatic theorem

Let $H(t)$ vary over $t \in [0, T]$, with instantaneous eigenstates $H(t) |E_k(t)\rangle = E_k(t) |E_k(t)\rangle$,
 $E_0 < E_1 \leq \dots$

Adiabatic theorem (informal)

Start in $|E_0(0)\rangle$. If $H(t)$ changes slowly enough *relative to the spectral gap*, then at time T the state is still close to the instantaneous ground state $|E_0(T)\rangle$.

Three hypotheses are doing real work, and all three can fail:

- **Start in the ground state:** easy, if $H(0)$ has a preparable one
- **A gap** $g(t) = E_1(t) - E_0(t) > 0$ throughout. If it closes, the theorem says nothing at all
- **A closed system:** no environment, no temperature. Real machines have both

E. Farhi, J. Goldstone, S. Gutmann, M. Sipser (2000). "Quantum Computation by Adiabatic Evolution." arXiv:quant-ph/0001106 (preprint, never journal-published)

What H_P looks like as a matrix

- Take the smallest Ising problem: two spins, $J_{12} = 1$, $h = 0$, so $H_P = -\sigma_1^z \sigma_2^z$

- The convention that links the two languages:

$$|0\rangle \leftrightarrow s = +1, \quad |1\rangle \leftrightarrow s = -1$$

basis state	spins (s_1, s_2)	$-J_{12}s_1s_2$	entry
$ 00\rangle$	$(+1, +1)$	-1	$H_{11} = -1$
$ 01\rangle$	$(+1, -1)$	$+1$	$H_{22} = +1$
$ 10\rangle$	$(-1, +1)$	$+1$	$H_{33} = +1$
$ 11\rangle$	$(-1, -1)$	-1	$H_{44} = -1$

$$H_P = -\sigma^z \otimes \sigma^z = \begin{pmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

Ground energy -1 , attained twice — by $|00\rangle$ and $|11\rangle$.

That is the spin-flip degeneracy of L3.

Switch on a field $h_1 = \frac{1}{2}$ and it splits:

$$\text{diag} \left(-\frac{3}{2}, \frac{1}{2}, \frac{3}{2}, -\frac{1}{2} \right)$$

now $|00\rangle$ alone is the ground state.

- Every off-diagonal entry is zero, because σ^z never moves a basis state — it only attaches a sign to it
- Read the matrix, and the algorithm is obvious.** The 2^n diagonal entries *are* the 2^n costs; the smallest one sits at the answer; and measuring in the computational basis returns a bit string you can price classically
- So H_P contains no quantum mechanics at all — it is the cost table of L2 written on a diagonal. What makes the next slide interesting is that H_B is *not* diagonal

Two Hamiltonians, and a line between them

Initial (driver) Hamiltonian

$$H_B = - \sum_{i=1}^n \sigma_i^x$$

Ground state $|+\rangle^{\otimes n}$: the uniform superposition over *all* candidates. One Hadamard per qubit.

Problem Hamiltonian

$$H_P = - \sum_{(i,j) \in E} J_{ij} \sigma_i^z \sigma_j^z - \sum_i h_i \sigma_i^z$$

Diagonal. Its ground state is the bit string we want.

The interpolation

$$H(s) = (1-s) H_B + s H_P \quad s = t/T$$

s is the *anneal fraction*.

- $s = 0$: every answer equally weighted
- $s = 1$: the ground state is the optimum
- in between: amplitude flows from “everything” to “the answer”

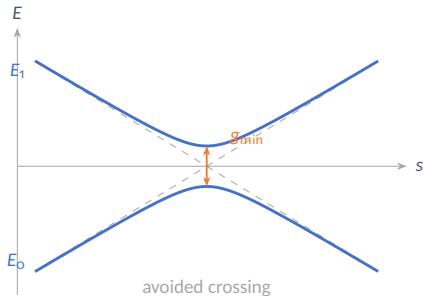
Nothing forces a straight line. Curved schedules $A(s)H_B + B(s)H_P$, and extra “catalyst” terms, are legitimate and sometimes much better.

The spectral gap is the whole story

$$g(s) = E_1(s) - E_0(s), \quad g_{\min} = \min_s g(s)$$

Levels that would generically cross instead *repel* — an **avoided crossing**. The closest approach is $g_{\min}$.

There the ground state changes character fastest. That is where the evolution must crawl, and where a fast sweep dumps population into $|E_1\rangle$.



Dashed: what a coupling-blind picture draws. Solid: what really happens.

How slow is slow enough?

- The adiabatic theorem turns into a runtime bound of the form $T \sim 1/g_{\min}^2$
- So everything depends on one question: how does the minimum gap behave as n grows?
- For easy instances $g_{\min}$ shrinks polynomially and the anneal is efficient
- For hard instances $g_{\min}$ is believed to close *exponentially*, $g_{\min} \sim 2^{-\alpha n}$, and T blows up with it

Why this is the honest picture

Exponentially small gaps are exactly what we should expect: if adiabatic evolution ran in polynomial time on every instance, it would put NP-hard problems in BQP.

The gap is where the hardness hides. It did not go away — it changed costume.

— T. Albash & D. A. Lidar (2018). “Adiabatic quantum computation.” *Rev. Mod. Phys.* 90, 015002. arXiv:1611.04471 · E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, D. Preda (2001). *Science* 292, 472–475.

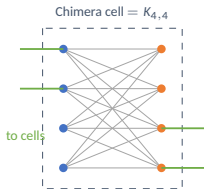
Adiabatic $\neq$ annealing

	Adiabatic QC (theory)	Quantum annealing (hardware)
System	closed, unitary	open, coupled to a thermal bath
Temperature	zero	~ 15 mK, but gaps can be smaller
Guarantee	adiabatic theorem	none
Universality	universal with the right H	stochastic H ; not universal
Output	the ground state	a <i>sample</i> , often excited

- D-Wave machines implement a transverse-field Ising anneal with real dissipation
- Thermal excitation can be *helpful* — it can kick the system out of a local minimum — but it removes any theoretical guarantee
- The right mental model is a *physically realised heuristic*, not a proof procedure

T. Kadowaki & H. Nishimori (1998). "Quantum annealing in the transverse Ising model." *Phys. Rev. E* 58, 5355. arXiv:cond-mat/9804280 · M. W. Johnson et al. (2011). *Nature* 473, 194–198.

Connectivity: Chimera, Pegasus, Zephyr



The chip is **not** a complete graph: only J_{ij} with a physical coupler exist.

Topology	Processor	Degree	Qubits (approx.)
Chimera	D-Wave 2000Q	6	2,000
Pegasus	Advantage	15	5,000
Zephyr	Advantage2	20	4,400

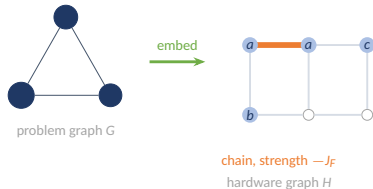
Degrees are exact; counts are round figures for shipping systems as of **early 2026**.

Each generation raises the degree, which is the number that matters — it decides how much of your problem graph survives without rewriting. Degree 20 is still far from the $n - 1$ a dense QUBO wants.

Minor embedding

Your problem graph G rarely sits inside the hardware graph H . **The fix:** represent one logical variable by a connected **chain** of physical qubits, tied together by strong ferromagnetic couplings $-J_F$ so they agree.

- Formally: find a *minor* of H isomorphic to G . NP-hard in general; production tools are heuristics.
- K_n costs $\Theta(n^2)$ physical qubits with chains of length $\Theta(n)$ — a 5,000-qubit chip holds a *dense* problem of only a few hundred variables.
- Embedding is classical work, and can take longer than the anneal. Benchmarks that



G has a triangle; the grid has none. Duplicating a buys the edge.

Chain strength and chain breaks

The trade-off in J_F

- **Too weak:** the chain's qubits disagree at readout — a *chain break*. The sample no longer decodes to a valid assignment.
- **Too strong:** J_F eats the coupler's fixed dynamic range. Since all $|J| \leq J_{\max}$, a large J_F pushes the real problem couplings down toward the analogue noise floor, and the machine solves a blurred problem.

Start near the RMS of the couplings incident on that variable, then sweep. Chain strength is a hyperparameter, and results are genuinely sensitive to it.

Repairing broken chains

Policy	Behaviour
Majority vote	commonest spin in the chain; ties at random
Discard	throw the whole sample away
Weighted	vote weighted by each qubit's local field
Minimize energy	greedily re-assign to lower H_P

The catch: repair is classical post-processing. Majority vote on a badly broken chain is a coin flip dressed as an answer — always report the **chain-break fraction**. Above a few percent you benchmark your decoder, not the annealer.

What annealing does not promise

The honest statement

There is **no proven general speedup** of quantum annealing over classical optimization heuristics for NP-hard problems.

- **First-order transitions.** For many hard families the path crosses a first-order quantum phase transition, at which $g_{\min}$ closes exponentially in n . The runtime is then exponential — the theorem satisfied, and useless.
- **Perturbative crossings.** Random Ising instances develop exponentially small gaps late in the anneal, between the true ground state and a distant local minimum. Invisible in the instances you can diagonalize.
- **Stochasticity.** The transverse-field Ising path has no sign problem, so quantum Monte Carlo can often simulate it. Not always — there are constructed counterexamples — but “no classical algorithm can do this” is not established.
- **Not universal.** Such an annealer is not known to be able to run Shor or Grover. Universality needs the general, non-stoquastic adiabatic model.

Summary

- **Hardness:** the problems of L1-L3 are NP-hard, and quantum computing is not expected to change that. NP-complete problems are not believed to be in BQP
- **Equivalence:** reductions let us target a single encoding. QUBO and Ising are that target, chosen because they are quadratic, unconstrained and diagonal
- **The price of the encoding:** constraints become penalties, penalties need slack variables, and slack variables need qubits
- **Annealing:** lets physics find the ground state, but the hardness reappears as an exponentially small spectral gap, and real hardware trades the guarantee for a thermal environment
- **What is still missing:** every method here either has no guarantee, or a guarantee too weak to make the problem tractable

References

Hardness and encodings

- A. Lucas (2014). "Ising formulations of many NP problems." *Frontiers in Physics* 2, art. 5. arXiv:1302.5843
- M. X. Goemans & D. P. Williamson (1995). "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming." *J. ACM* 42(6), 1115–1145. doi:10.1145/227683.227684
- S. Khot, G. Kindler, E. Mossel, R. O'Donnell (2007). "Optimal inapproximability results for MAX-CUT and other 2-variable CSPs?" *SIAM J. Comput.* 37(1), 319–357.

Adiabatic evolution and quantum annealing

- T. Kadowaki & H. Nishimori (1998). "Quantum annealing in the transverse Ising model." *Phys. Rev. E* 58, 5355. arXiv:cond-mat/9804280
- E. Farhi, J. Goldstone, S. Gutmann, M. Sipser (2000). "Quantum Computation by Adiabatic Evolution." arXiv:quant-ph/0001106 (preprint)
- E. Farhi, J. Goldstone, S. Gutmann, J. Lapan, A. Lundgren, D. Preda (2001). "A quantum adiabatic evolution algorithm applied to random instances of an NP-complete problem." *Science* 292, 472–475.
- T. Albash & D. A. Lidar (2018). "Adiabatic quantum computation." *Rev. Mod. Phys.* 90, 015002. arXiv:1611.04471
- M. W. Johnson et al. (2011). "Quantum annealing with manufactured spins." *Nature* 473, 194–198.
- V. Choi (2008, 2011). "Minor-embedding in adiabatic quantum computation, I and II." *Quantum Inf. Process.* 7, 193 / 10, 343. arXiv:0804.4884, arXiv:1001.3116

Benchmarking, and the current state of play

- A. Abbas et al. (2024). "Challenges and opportunities in quantum optimization." *Nature Reviews Physics* 6, 718–735. arXiv:2312.02279
- A. D. King et al. (2025). "Beyond-classical computation in quantum simulation." *Science* 388, 199–204. arXiv:2403.00910